Week 3 Lesson 2 — Naive Bayes applied to Restaurant Reviews

2nd November 2026

1 Naive Bayes: restaurant review sentiment

This notebook builds a classifier step by step using **CountVectorizer** and **MultinomialNB**. Run the cells from top to bottom using your existing student environment. No installation or upgrades are needed.

You will see where the probabilities come from, inspect word counts, classify new reviews, and investigate unknown words and sarcasm.

The 12 training examples are deliberately small for learning. Their results are not a measure of real-world accuracy, and model probabilities are not guaranteed confidence levels.

[1]:
```
import numpy as np
import pandas as pd
import sklearn
from IPython.display import display
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB

print("scikit-learn version:", sklearn.__version__)
```

```
scikit-learn version: 1.5.1
```

1.1 1. Start with labelled examples

A person has already assigned each training review a sentiment. The model learns from these labels; it does not start with a dictionary of positive and negative words.

[2]:
```
training = pd.DataFrame([
    ("delicious food and great service", "positive"),
    ("excellent food and friendly staff", "positive"),
    ("great meal delicious food", "positive"),
    ("lovely restaurant excellent service", "positive"),
    ("fresh food and wonderful service", "positive"),
    ("friendly staff and delicious meal", "positive"),
    ("terrible food and rude service", "negative"),
    ("cold food and slow service", "negative"),
    ("awful meal rude staff", "negative"),
    ("overpriced restaurant terrible service", "negative"),
    ("bland food and awful service", "negative"),
```

```
    ("slow staff and cold meal", "negative"),
], columns=["review", "sentiment"])
display(training)
```

```
                                  review sentiment
0         delicious food and great service  positive
1        excellent food and friendly staff  positive
2                great meal delicious food  positive
3      lovely restaurant excellent service  positive
4         fresh food and wonderful service  positive
5        friendly staff and delicious meal  positive
6           terrible food and rude service  negative
7               cold food and slow service  negative
8                    awful meal rude staff  negative
9   overpriced restaurant terrible service  negative
10            bland food and awful service  negative
11               slow staff and cold meal  negative
```

1.2 2. Turn text into word counts

`fit_transform` learns the vocabulary from the training reviews and counts each word. Each row is a review; each column is a word. Repeated words have larger counts. Word order is discarded.

We keep common words here so the calculation remains easy to inspect.

[3]:
```
vectorizer = CountVectorizer()
X_train = vectorizer.fit_transform(training["review"])
words = vectorizer.get_feature_names_out()
word_table = pd.DataFrame(X_train.toarray(), columns=words)
display(word_table)
print("Reviews × vocabulary:", X_train.shape)
```

```
    and  awful  bland  cold  delicious  excellent  food  fresh  friendly  \
0     1      0      0     0          1          0     1      0         0
1     1      0      0     0          0          1     1      0         1
2     0      0      0     0          1          0     1      0         0
3     0      0      0     0          0          1     0      0         0
4     1      0      0     0          0          0     1      1         0
5     1      0      0     0          1          0     0      0         1
6     1      0      0     0          0          0     1      0         0
7     1      0      0     1          0          0     1      0         0
8     0      1      0     0          0          0     0      0         0
9     0      0      0     0          0          0     0      0         0
10    1      1      1     0          0          0     1      0         0
11    1      0      0     1          0          0     0      0         0

    great  lovely  meal  overpriced  restaurant  rude  service  slow  staff  \
0       1       0     0           0           0     0        1     0      0
1       0       0     0           0           0     0        0     0      1
```

```
2       1       0     1           0           0     0       0     0       0
3       0       1     0           0           1     0       1     0       0
4       0       0     0           0           0     0       1     0       0
5       0       0     1           0           0     0       0     0       1
6       0       0     0           0           0     1       1     0       0
7       0       0     0           0           0     0       1     1       0
8       0       0     1           0           0     1       0     0       1
9       0       0     0           1           1     0       1     0       0
10      0       0     0           0           0     0       1     0       0
11      0       0     1           0           0     0       0     1       1

    terrible  wonderful
0          0          0
1          0          0
2          0          0
3          0          0
4          0          1
5          0          0
6          1          0
7          0          0
8          0          0
9          1          0
10         0          0
11         0          0

Reviews × vocabulary: (12, 20)
```

1.3 3. Train Naive Bayes

`fit` learns the fraction of reviews in each class (the prior) and the word frequencies within each class (the likelihoods).

With `alpha=1.0`, add-one smoothing gives each vocabulary word an extra count in each class:

$$P(w \mid c) = \frac{\text{count of word } w \text{ in class } c + 1}{\text{total word count in class } c + |V|}$$

Here the likelihood is for a **word occurrence**, not the fraction of reviews containing the word. This is the multinomial version of Naive Bayes.

[4]:
```
model = MultinomialNB(alpha=1.0)
model.fit(X_train, training["sentiment"])

display(pd.DataFrame({
    "class": model.classes_,
    "training reviews": model.class_count_.astype(int),
    "prior": np.exp(model.class_log_prior_),
}))
```

```
likelihoods = pd.DataFrame(
    np.exp(model.feature_log_prob_).T,
    index=words,
    columns=model.classes_,
)
likelihoods.index.name = "word"
display(likelihoods.loc[["delicious", "food", "great", "rude", "terrible"]])
```

```
      class  training reviews  prior
0  negative                 6    0.5
1  positive                 6    0.5
```

```
           negative  positive
word
delicious  0.020833  0.083333
food       0.083333  0.104167
great      0.020833  0.062500
rude       0.062500  0.020833
terrible   0.062500  0.020833
```

1.3.1 Why does "delicious" have a nonzero negative probability?

"Delicious" appears **3 times in the positive training reviews and 0 times in the negative reviews**. Its nonzero negative probability comes from **add-one (Laplace) smoothing**, enabled by `MultinomialNB(alpha=1.0)`.

There are **28 word occurrences in each class** and **20 distinct words in the vocabulary**. Smoothing adds one pretend occurrence (a pseudocount) of every vocabulary word to each class, so each denominator becomes `28 + 20 = 48`.

Class	Actual "delicious" count	Smoothed probability
Negative	0	`(0 + 1) / 48 = 0.020833`
Positive	3	`(3 + 1) / 48 = 0.083333`

The negative value is therefore exactly **one quarter** of the positive value: the smoothed counts are 1 and 4, with equal denominators.

Why smooth? Without smoothing, the zero negative likelihood for "delicious" would make the entire negative score zero whenever that word appears, regardless of the other words—even in "delicious food but terrible service." Smoothing keeps unseen combinations possible; absence from this tiny training sample is not proof that a word can never occur in that class.

These table entries are **P(word | class)**: probabilities of word occurrences given a class. They are **not P(class | word)**, the probability of a class given a word. Here, each occurrence of "delicious" multiplies the positive-to-negative score ratio by four; the other words and the class priors also contribute to the final prediction.

1.4 4. Predict new reviews

For each class, the model combines its prior with the likelihoods of the known words. It then normalizes the class scores so their probabilities sum to 1.

These reviews are demonstrations, not an independent benchmark dataset.

[5]:
```
new_reviews = [
    "delicious food and great service",
    "cold food and rude staff",
    "tasty food and great service",
    "tasty",
    "the food had no salt how delicious it was",
]
X_new = vectorizer.transform(new_reviews)  # Reuse the training vocabulary.
results = pd.DataFrame({
    "review": new_reviews,
    "prediction": model.predict(X_new),
})
for i, label in enumerate(model.classes_):
    results[f"P({label} | words)"] = model.predict_proba(X_new)[:, i]
display(results.style.format({
    f"P({label} | words)": "{:.1%}" for label in model.classes_
}))
```

```
<pandas.io.formats.style.Styler at 0x117a10620>
```

1.5 5. What happens to “tasty”?

The vectorizer ignores words outside its training vocabulary. Smoothing applies to **known vocabulary words** that may be missing from one class; it does not teach the meaning of an unseen word.

For a review consisting only of “tasty”, there are no known words. The prediction probabilities equal the class priors: 50% each here. The returned label simply breaks the tie; it is not evidence of sentiment.

[6]:
```
analyzer = vectorizer.build_analyzer()
for review in new_reviews:
    tokens = analyzer(review)
    known = [word for word in tokens if word in vectorizer.vocabulary_]
    unknown = sorted(set(tokens) - set(vectorizer.vocabulary_))
    print(review)
    print("  Known:", known)
    print("  Ignored:", unknown)

assert "tasty" not in vectorizer.vocabulary_
assert vectorizer.transform(["tasty"]).nnz == 0
np.testing.assert_allclose(
    model.predict_proba(vectorizer.transform(["tasty"]))[0],
```

```
    np.exp(model.class_log_prior_),
)
```

```
delicious food and great service
  Known: ['delicious', 'food', 'and', 'great', 'service']
  Ignored: []
cold food and rude staff
  Known: ['cold', 'food', 'and', 'rude', 'staff']
  Ignored: []
tasty food and great service
  Known: ['food', 'and', 'great', 'service']
  Ignored: ['tasty']
tasty
  Known: []
  Ignored: ['tasty']
the food had no salt how delicious it was
  Known: ['food', 'delicious']
  Ignored: ['had', 'how', 'it', 'no', 'salt', 'the', 'was']
```

1.6 6. Inspect one prediction by hand

For "great food", calculate a score for each class:

$$\text{score}(c) = P(c)P(\text{great} \mid c)P(\text{food} \mid c)$$

Dividing each score by the sum of both scores produces the posterior probabilities under this model. For longer reviews, the library uses log probabilities to avoid extremely small products.

[7]:
```
example = "great food"
manual = pd.DataFrame(index=model.classes_)
manual["prior"] = np.exp(model.class_log_prior_)
manual["P(great | class)"] = likelihoods.loc["great"]
manual["P(food | class)"] = likelihoods.loc["food"]
manual["score"] = manual.prod(axis=1)
manual["posterior"] = manual["score"] / manual["score"].sum()
display(manual)

library_probabilities = model.predict_proba(vectorizer.transform([example]))[0]
np.testing.assert_allclose(manual["posterior"].to_numpy(),␣
  ↪library_probabilities)
print("Our calculation matches scikit-learn.")
```

	prior	P(great | class)	P(food | class)	score	posterior
negative	0.5	0.020833	0.083333	0.000868	0.210526
positive	0.5	0.062500	0.104167	0.003255	0.789474

```
Our calculation matches scikit-learn.
```

1.7 7. Why sarcasm fails here

In “the food had no salt how delicious it was”, this training vocabulary recognizes only “food” and “delicious”. It has never learned “no” or “salt”. The known words lead it toward positive sentiment.

Even with a larger vocabulary, this bag-of-words model discards word order and cannot reliably interpret irony. Learning pairs of words (`ngram_range=(1, 2)`) can capture some short phrases, but does not solve sarcasm.

1.8 8. Try your own review

Edit the text below and rerun the cell. To teach the model new vocabulary, add labelled examples to `training` and rerun from the vectorizer training cell onward. A real evaluation needs separate labelled test data that was not used to fit the vocabulary or classifier.

[8]:
```
my_review = "great food but rude staff"
probabilities = model.predict_proba(vectorizer.transform([my_review]))[0]
print("Review:", my_review)
print("Prediction:", model.predict(vectorizer.transform([my_review]))[0])
display(pd.Series(probabilities, index=model.classes_, name="model␣
 ↪probability"))
```

```
Review: great food but rude staff
Prediction: positive
```

```
negative    0.444444
positive    0.555556
Name: model probability, dtype: float64
```

1.9 References

Official scikit-learn 1.5 documentation:

- CountVectorizer
- MultinomialNB